

Balance Machine Learning Task Report



Cağla Ipek Koçal

Table of Contents

1. Project Summary.....	1
2. Motivation and Novelty	2
3. Big Picture	3
4. Methods and Implementation Details	4
5. Results, Discussions, and Future Directions.....	9
5.1. Results and Discussion.....	9
5.2. Future Directions.....	23
6. Further Discussion	24

1. Project Summary

In this project, my primary aim was to forecast the daily balance values for various bank accounts, considering the nuances of inflow/outflow and other pivotal account-related parameters. The methodology I embraced was broken down into two main segments.

The initial segment revolves around a static database where the transactional data for the accounts resides. The SQL code for creating the database was given for the task but it required some additional debugging and adjustments. After I managed to read the SQL code into a database, I read the data into a pandas data frame and then started the necessary preprocessing steps to prepare the data into a suitable form for machine learning.

The second crucial segment is my machine learning framework. Crafted meticulously, this framework is designed to draw essential insights from the input data. A sequence of processes is then engaged, including training using a diverse collection of ML models, their evaluation via time-series cross-validation, and enabling forecasting using the chosen model. Although the framework does not work in an end-to-end manner right now it will be developed into an end-to-end form which is integrable to a user interface in later stages. In this phase of the project, fulfilling the requirements of my task, I have used two machine learning models and successfully tuned them to recognize the specific patterns in the data provided.

Throughout the project's lifecycle, I dabbled with a plethora of machine learning models. Gradient boosting methods like LightGBM and, linear models such as linear regression were tried but I ventured deeper and found out that autoregressive algorithms like SARIMAX and neural network constructs like multi-layer perceptrons works much better in this task. After the selection of the models I will use, I tuned these models and tested their performance in a separate testing data.

Python emerged as my language of choice for this initiative. For the construction of the machine learning framework, I utilized the capabilities of the VS Code integrated development environment and used PostgreSQL for my database management system.

Wrapping up the project, I derived immense satisfaction from the capability of such system and I anticipate a large room for further development of this project. My machine learning framework, with its precision-driven approach, could achieve Root Mean Squared Error scores as low as 2614.24 for daily account balance forecasting over a 30-day timeframe. For three month balance forecasting, the framework demonstrated 3652.79 RMSE.

2. Motivation and Novelty

As financial markets fluctuate, having a robust forecasting model in place allows account holders and financial institutions to adjust their transactional patterns in real-time, resulting in a continuous flow of funds and financial stability. An accurate balance forecasting model facilitates the long-term stability and sustainability of banking and financial institutions. It becomes pivotal for strategic decision-making concerning liquidity management, risk assessment, operational efficiency, and financial planning.

Moreover, precise forecasts for account balances can be leveraged to craft financial strategies, ensuring financial stability and liquidity for the institution. Clients with access to accurate forecasting data can better manage their expenditure during high transaction periods, ensuring financial stability and avoiding potential shortfalls. In today's world, various factors drive the global financial dynamics - evolving financial markets, rising consumer demands, rapid technological advancements, and the need for sustained economic growth. Given this backdrop, the effective management of financial resources demands a robust forecasting strategy.

However, forecasting in the financial sector isn't straightforward. Crafting a reliable forecast often hinges on identifying and integrating crucial information. If the data pool is sparse, the forecast might be skewed. Conversely, excessive, or irrelevant data can make the model overly complex, leading to biases. While intricate models may offer a high degree of accuracy, they can be challenging to maintain. Simpler, slightly less accurate models are often preferred, especially when the forecast module is a subset of a larger financial tool. Therefore, selecting the right forecasting strategy is paramount.

Within the scope of this project, I sought to explore the optimal forecasting strategy for several account balances. I wanted to predict, one month ahead and three month ahead, account balances, integrating this strategy into a comprehensive machine learning framework.

3. Big Picture

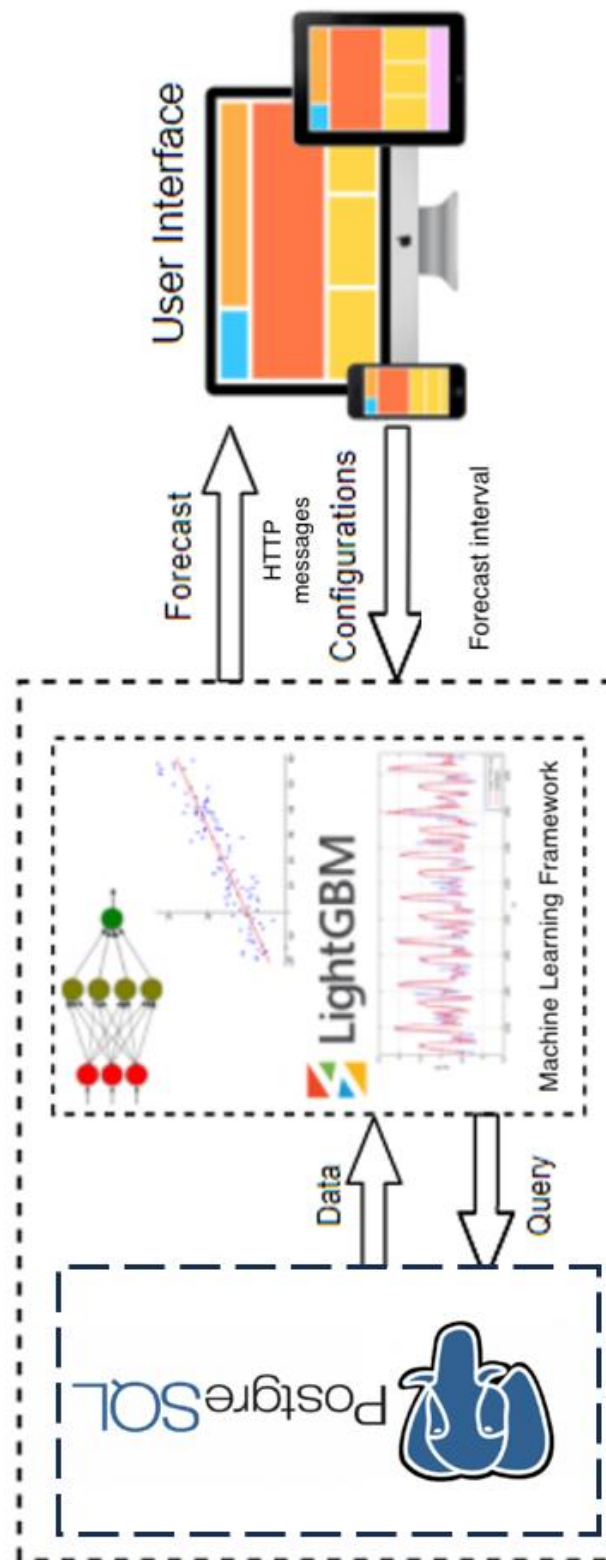


Figure 1: The "Big Picture" of the project(Need for a Web Server is anticipated)

4. Methods and Implementation Details

The first segment of the project is primarily designed for data preprocessing and visualization of financial transactions based on account IDs. Initially, it filters the dataset to exclude any rows marked as **is_deleted** and further narrows it down to include only 'INFLOW' and 'OUTFLOW' transaction directions. This ensures the dataset contains only relevant and active transactions.

A crucial preprocessing step is the conversion of the **date** column from seconds to a more readable datetime format using **pd.to_datetime**, aiding in subsequent time-series operations.

The dataset is then grouped by **account_id**, facilitating individualized analysis for each account. For every unique account, the transactions are aggregated by date and type (either 'INFLOW' or 'OUTFLOW'). These aggregated transactions are merged into a consolidated DataFrame, ensuring a coherent representation of both incoming and outgoing amounts.

To visualize this data, the **plot_daily_inflow_outflow** function is introduced. This function generates bar charts depicting daily transaction amounts for a specified account ID. Such visualization provides a clear picture of the financial inflow and outflow for an account over time.

Lastly, the code accounts for potential data irregularities. Given that financial data may not always be recorded for every day, it's vital to address missing values. The code resamples the data on a daily frequency and employs a forward-fill method to substitute any missing 'INFLOW' or 'OUTFLOW' values with the last recorded value, ensuring continuity in the dataset.

The second part of the project which is development of Machine Learning Models and creating an end-to-end machine learning framework(as a future direction), includes some essential steps which can be summarized as follows:

The exploratory data analysis is essential as it is beneficial for selecting necessary features. Visualizing the correlations between each feature and the target variable can make the selection process easier. Moreover, feature engineering is at the core of every machine learning project. The collected data should be reshaped into the desired form before feeding it as an input to the machine learning algorithm. Many state-of-the-art machine learning algorithms require the standardization of the features to avoid problems such as the exploding gradient problem. Feature engineering is also critical to provide inputs to the learning algorithms in a suitable form. Learning algorithms usually need features derived from the primary data to extract

information about the target data. Such features include lagged values, rolling means, and rolling standard deviations of the target variable or the side information (day of the week, whether it is weekday or not). Reformations of the target data and selection of the main features are also a part of the feature engineering process. Selecting which data to use as side information is vital since highly correlated data to the target variable may drastically increase the model's accuracy. It is also essential to manipulate the target data and reform it in a plausible form, primarily since we work on time series data which violates the data being identically distributed and independent assumption of many machine learning algorithms. Time-series data involves the correlation between adjacent data instances, which makes them dependent. However, such dependency can be eliminated by feature engineering methods such as differencing or creating the lag or rolling window features making the data stationary. At the end of these considerations, I have started to integrate the features I want to use:

In order to implement the features to be used in the training and prediction process, they have been chosen by looking at similar examples from Kaggle competitions. In order to implement the features, a function has been written in order to merge them in a framework format. The function does the following: adds the lag ,rolling mean, rolling standard deviation for chosen number of days. For instance at the running file if there is a "lag-28" value then our function shifts the balance value for 28 days and adds as a separate columns for the feature data frame. Using this function I implemented the features ["lag-1", "lag-2", "lag-3", "lag-4", "lag-5", "lag-6", "lag-7", "lag-8", "lag-9", "lag-10", "last-5days-mean", "last-6days-mean", "last-7days-mean", "last-30days-mean", "last-7days-std", "last-30days-std", "averagevalue-30-31", "is_weekend", "day_of_week"] which can explained as the follows. The "lag-i" feature is obtained by shifting the data by i days and adding as a separate column. The "last-i-days-mean" features is obtained by taking the mean of the last i days of the data and the "last-i-days-std" by taking the standard deviation of the last i days of the data. These features helps us to transform the data into Independent and identically distributed random variables as a forecasting problem does not obey this condition. However, with these ways of standardizing, our data becomes independent of time and hence it is possible to implement machine learning algorithms to predict.

After choosing these features as our initial features, I plotted the feature importance maps that are taken from the LightGBM algorithm's attribute that is based on how much each feature

being used and then dropped the features that has lower importance. The feature importance map and the selected features can be found below:

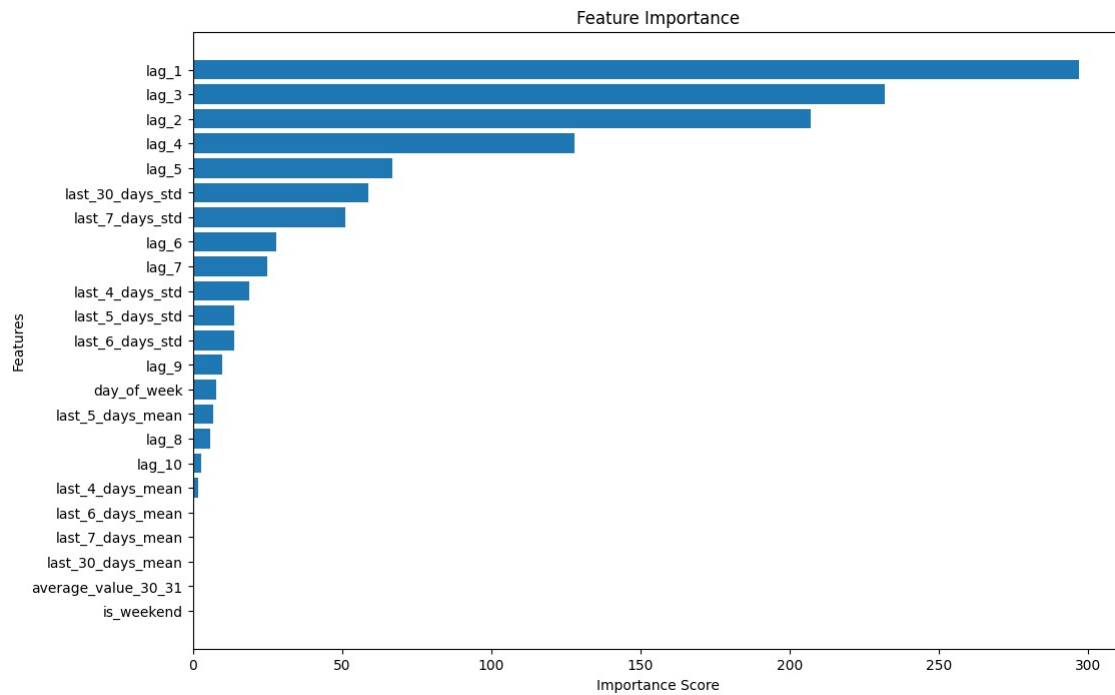


Figure 2: Feature importance map of all features

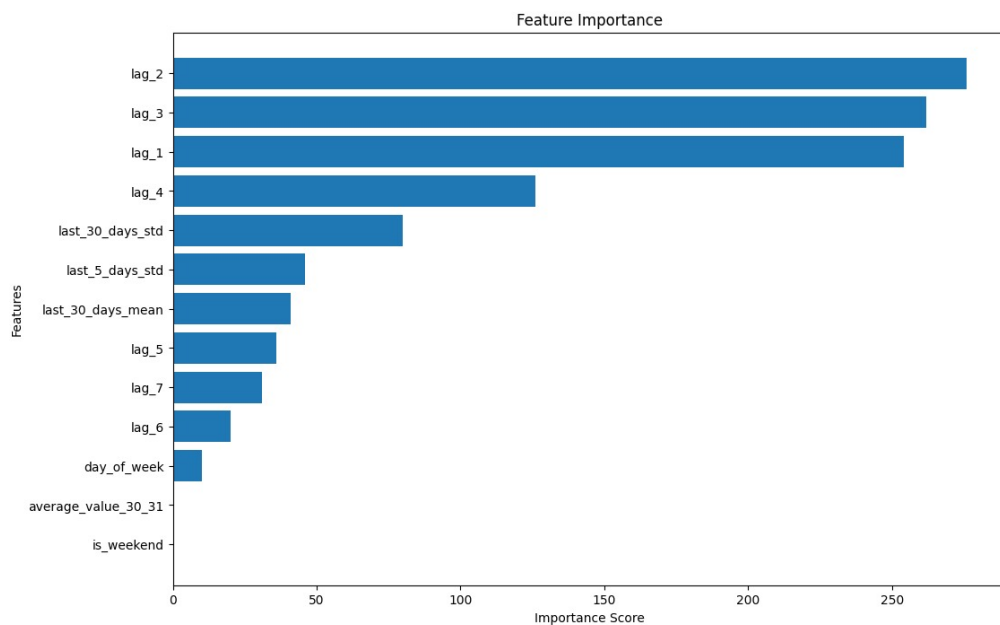


Figure 3: Feature importance map of selected features

Furthermore, **the construction and training of each machine learning algorithm** require elaborate work and careful attention. Each machine learning algorithm has unique qualities and will capture linear or nonlinear patterns between features and target variables differently. It is essential to know each of these algorithms to avoid mistakes while implementing them and

avoid misconceptions while interpreting their performances. These machine learning algorithms, as mentioned earlier, were used for predicting both the net balance and inflow and outflow values independently. In this stage, I test data with different models and compare their performances. To this end, I decided to use neural networks and SARIMAX, algorithms to fit the data.

SARIMAX is an auto-regressive algorithm, and it also uses lag values mentioned previously. It is a very traditional algorithm that is used for these tasks even before machine learning (with its today's definition) was not existing. Moreover, I chose to use neural networks since this model is quite helpful in determining non-linear relations in data. With this specification, outperforming simple models like linear regression would not be surprising for this algorithm since our data have this type of relation in it. One key advantage of neural networks is the feature extraction done within the algorithm. Effects of the features can sometimes be unpredictable and determining them manually can be extremely tedious. Neural networks do it automatically according to the dataset. Neural networks generally consist of three layers: input layer, hidden layer, and output. The number of hidden layers depends on the complexity of the model and can be adjusted in case of overfitting. Also, in networks, we have a chance to drop out some weights, which will also be helpful if we do not want our model to learn too much from a feature. However, it is beneficial to keep in mind that we don't have too much data so we can't train deep networks since they would overfit the training data. However, we can utilize the Multilayer Perceptron with few layers for our needs.

Another vital step is **hyperparameter tuning** which enables us to acquire maximum efficiency from each model. Each of these models are expected to be fine-tuned. To achieve this, it is expected to know the parameters of each model very well to understand which of these may be used to improve the model's accuracy. **Cross-validation concept**, is also crucial, which is a highly beneficial method for validating the model's parameters, especially when the dataset is not very large. Two cross-validation techniques, namely time series cross-validation and regular k-fold cross-validation, are evaluated in this project to enhance the performance of our models. However, after some literature review, I decided to use Time-series cross-validation as K-fold cross validation can cause data leakage in time series data. In other words, it led us to train our data in a time interval that is later than the time interval that we use for testing the data which can be misleading as this can bias our models and give predictions that are falsely accurate. The last criterion, forecasting for different periods, is also important and complicated. Since we want long periods of forecasts, I chose 30-day-ahead and 90-day-ahead as my two

prediction intervals. For the Neural Network model, such long forecasting periods require strategies like recursive methods and naturally give worse results than 1 day ahead predictions because of error accumulation. The recursive strategy includes using a prediction as if it is the ground truth for predictions of balance values from later days. The procedure for this strategy can be summarized by the following steps:

1. **Start with a Model:** First, you'll need a forecasting model, whether it's SARIMA, a neural network, LightGBM, or any other time series forecasting model.
2. **Train on Available Data:** You initially train your model using all the available data up to the current point in time.
3. **One-step Ahead Forecast:** Once trained, you use the model to predict the next data point (let's say, the next day's value).
4. **Shift and Update:** Now, for predicting two days ahead, you take the predicted value for the first day and add it to the end of your training dataset, essentially treating your forecasted value as a true value. This effectively extends your dataset by one day.
5. **Iterate:** You then use this updated dataset (which now includes your first forecasted value) to forecast the day after that. This process is repeated for however many days into the future you want to forecast.
6. **Potential Drawback:** An essential thing to remember is that errors can accumulate. Since each subsequent forecast uses previous forecasted values, if one prediction is off, the next one is more likely to be off too, and this error can accumulate as forecasting further into the future.

The Gantt chart for the progress of the project can be found below:

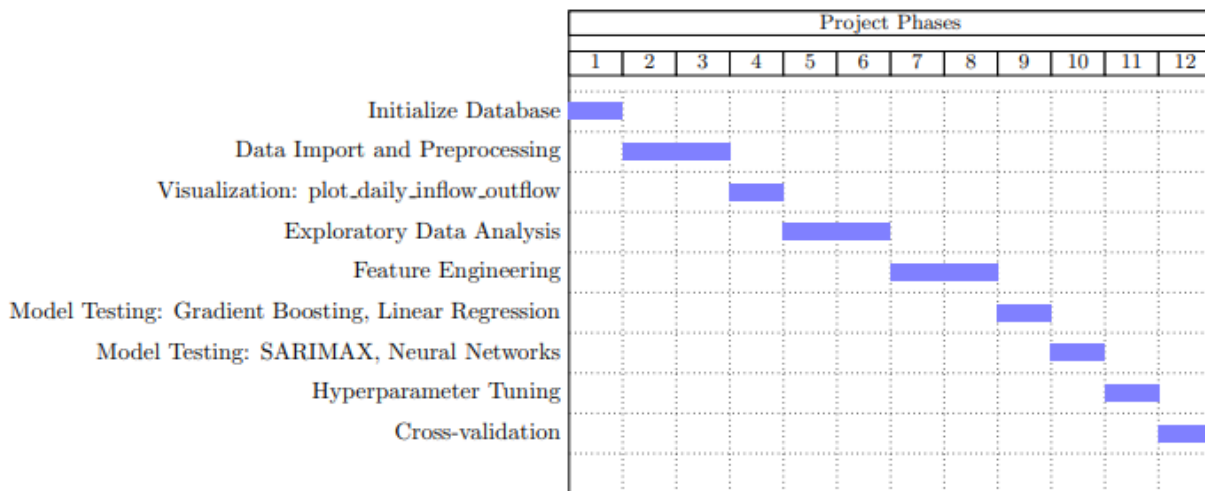
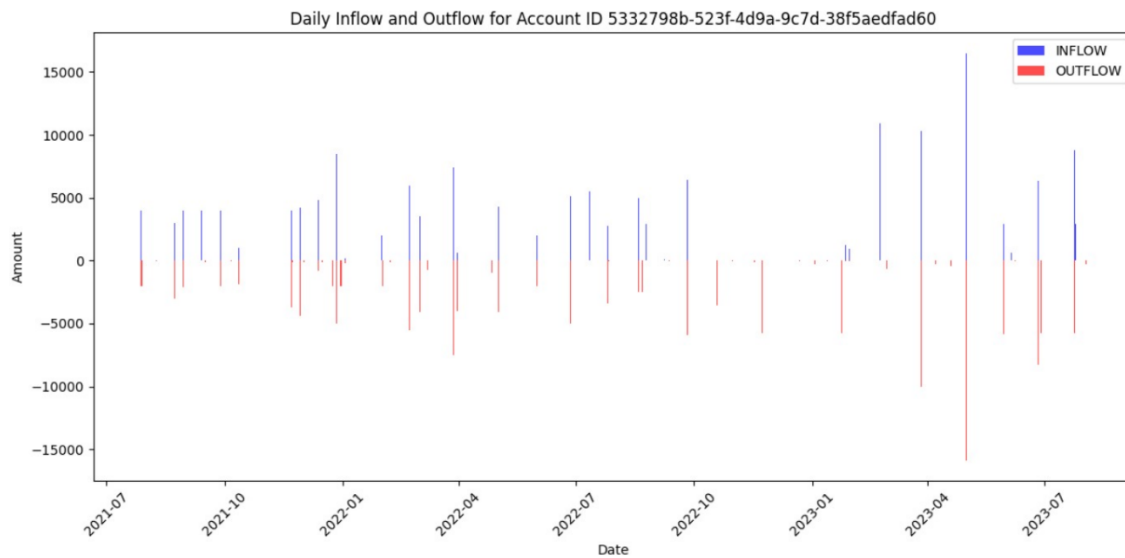


Figure 4: The Gantt chart of the project

5. Results, Discussion and Future Directions:

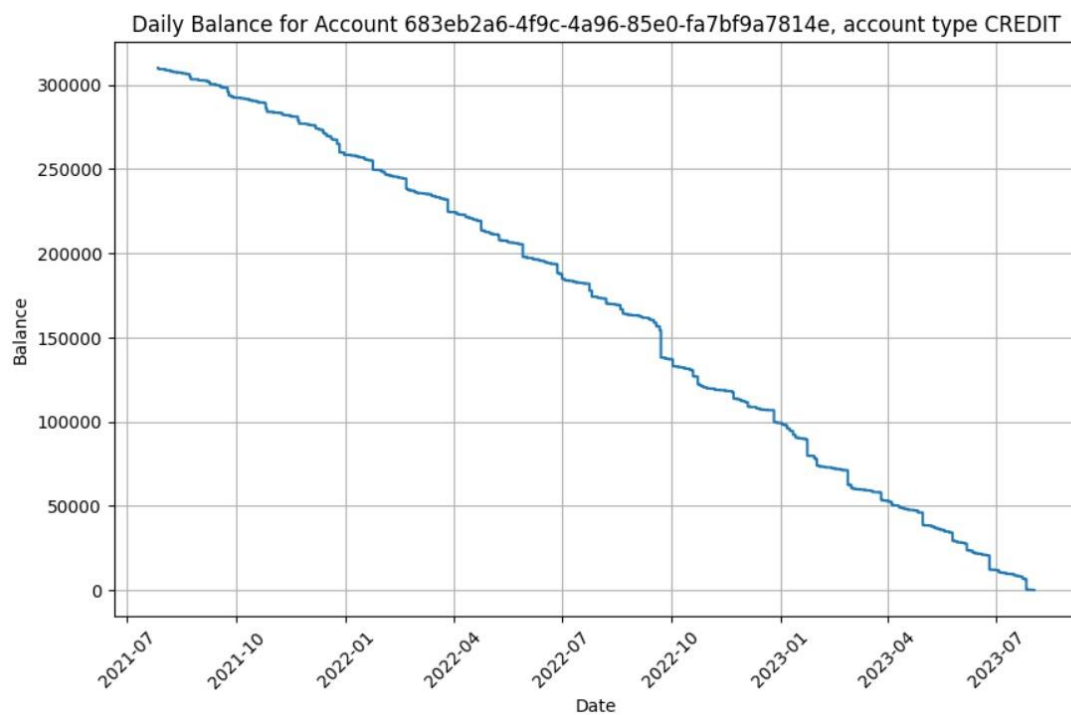
5.1 Results and Discussion:

The daily inflow and outflows of a selected account can be found below:



	Datetime	INFLOW	OUTFLOW
0	2021-07-27	304.07	-352.40
1	2021-07-28	0.00	-32.22
2	2021-07-29	0.00	-47.75
3	2021-07-30	0.00	-49.25
4	2021-07-31	0.00	0.00
5	2021-08-01	0.00	-143.37
6	2021-08-02	0.00	-342.58
7	2021-08-03	0.00	-168.23
8	2021-08-04	0.00	-66.00
9	2021-08-05	0.00	-95.85
10	2021-08-06	0.00	-340.63
11	2021-08-07	0.00	0.00
12	2021-08-08	0.00	-461.02
13	2021-08-09	0.00	0.00
14	2021-08-10	0.00	-56.58
15	2021-08-11	0.00	-161.13

The daily balance for a chosen account is obtained as follows:

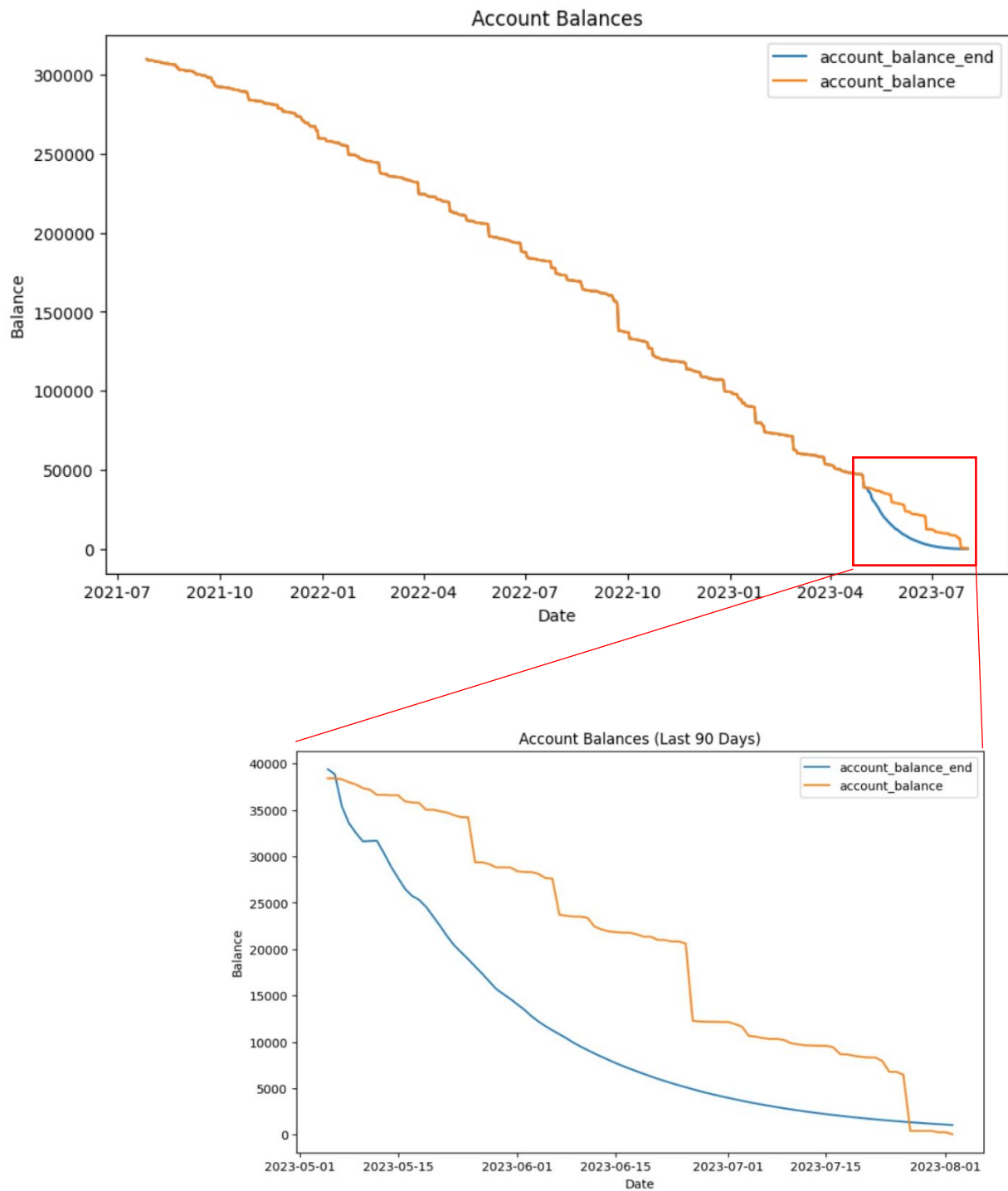


	date	balance
0	2021-08-02	-899.61
1	2021-08-03	-899.61
2	2021-08-04	-899.61
3	2021-08-05	-899.61
4	2021-08-06	-899.61
..	...	...
727	2023-07-30	500.45
728	2023-07-31	500.51
729	2023-08-01	500.51
730	2023-08-02	500.51
731	2023-08-03	500.57

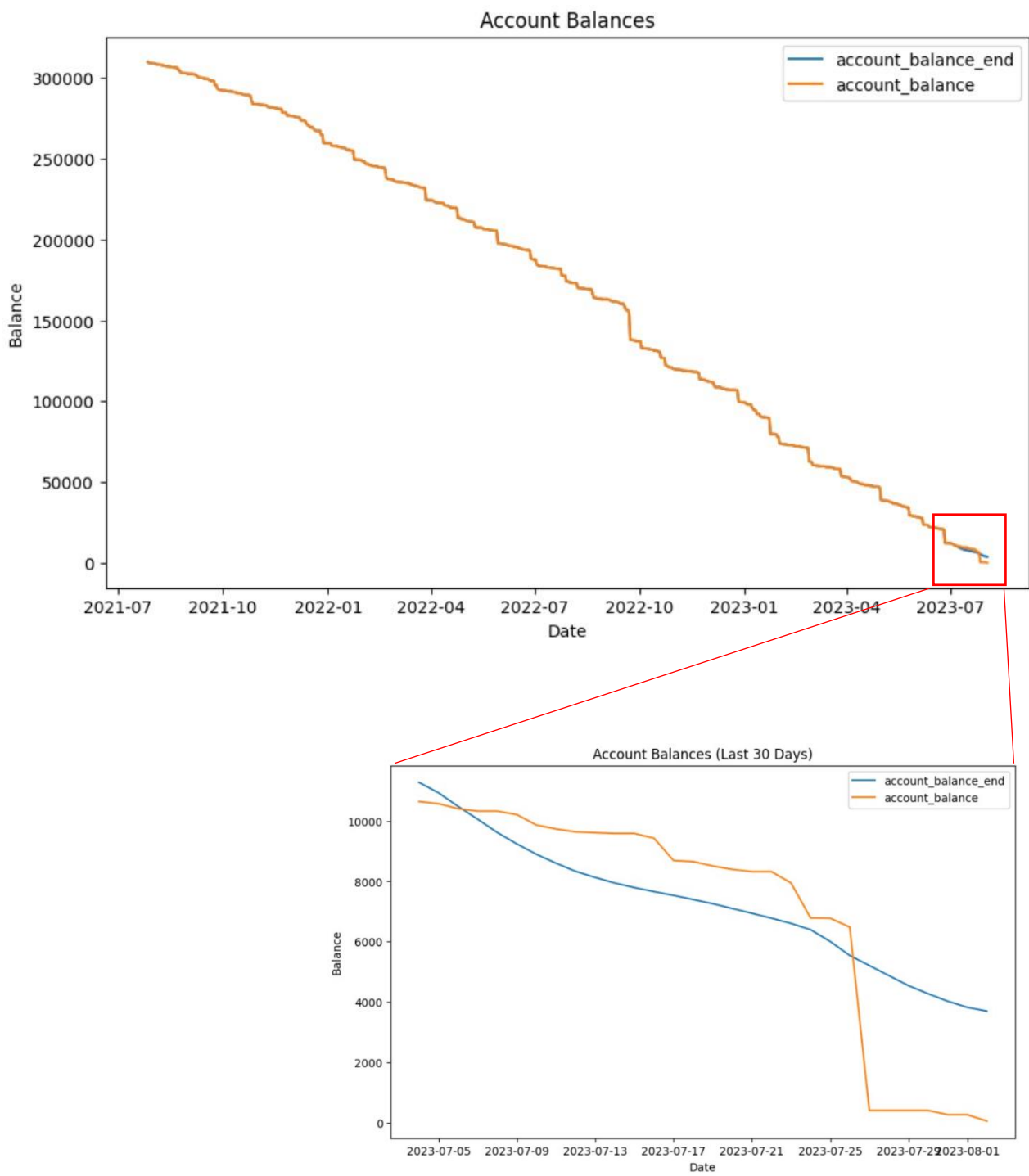
[732 rows x 2 columns]

After obtaining the daily balance values and processing the data as described previously, I have made 2 types of predictions which are 30 day ahead and 90 day ahead predictions. The key strategy for both prediction types is to use the previously described recursive method. One important detail to mention is that for 30 day predictions I have also tested in a 90 day period. However, this is simply for better generalizability of test data and actually for after every 30 day period I gave the actual values and break the recursive cycle for this type of strategy. Another way to think about this evaluation method is to think about the 1 day ahead predictions and how their evaluation is made. In one day ahead predictions, the last one day of the test data cannot be allocated as the test dataset since this can lead to generalizability issues. Hence, one day ahead forecasts are made for a longer interval with having the previous days balances available for each days forecast. The same also applies for this case. The predictions are still 30 day ahead but they simply being evaluated in a 90 day interval. In a more mathematical definition, let's say that we receive data $\{y(1), y(2), y(3), \dots\}$ and corresponding side information sequence $\{s(1), s(2), \dots\}$, where $y(t) \in \mathbb{R}$ and $s(t) \in \mathbb{R}^d \forall t \in \{1, 2, \dots\}$, f is our machine learning model and $\hat{y}_k(t+1)$ is our prediction for time $t+1$. While a one day ahead forecast is formulated as $\hat{y}_k(t+1) = f(\{y(1), \dots, y(t)\}, \{s(1), \dots, s(t)\})$ a one month ahead prediction would be $\{\hat{y}_k(t+1) = f(\{y(1), \dots, y(t)\}, \{s(1), \dots, s(t)\}), \hat{y}_k(t+2), f(\{y(1), \dots, y(t)\}, \{s(1), \dots, s(t)\}), \dots, \hat{y}_k(t+30) = f(\{y(1), \dots, y(t)\}, \{s(1), \dots, s(t)\})\}$. For a one month ahead prediction with 90 day interval, this set is predicted for three times.

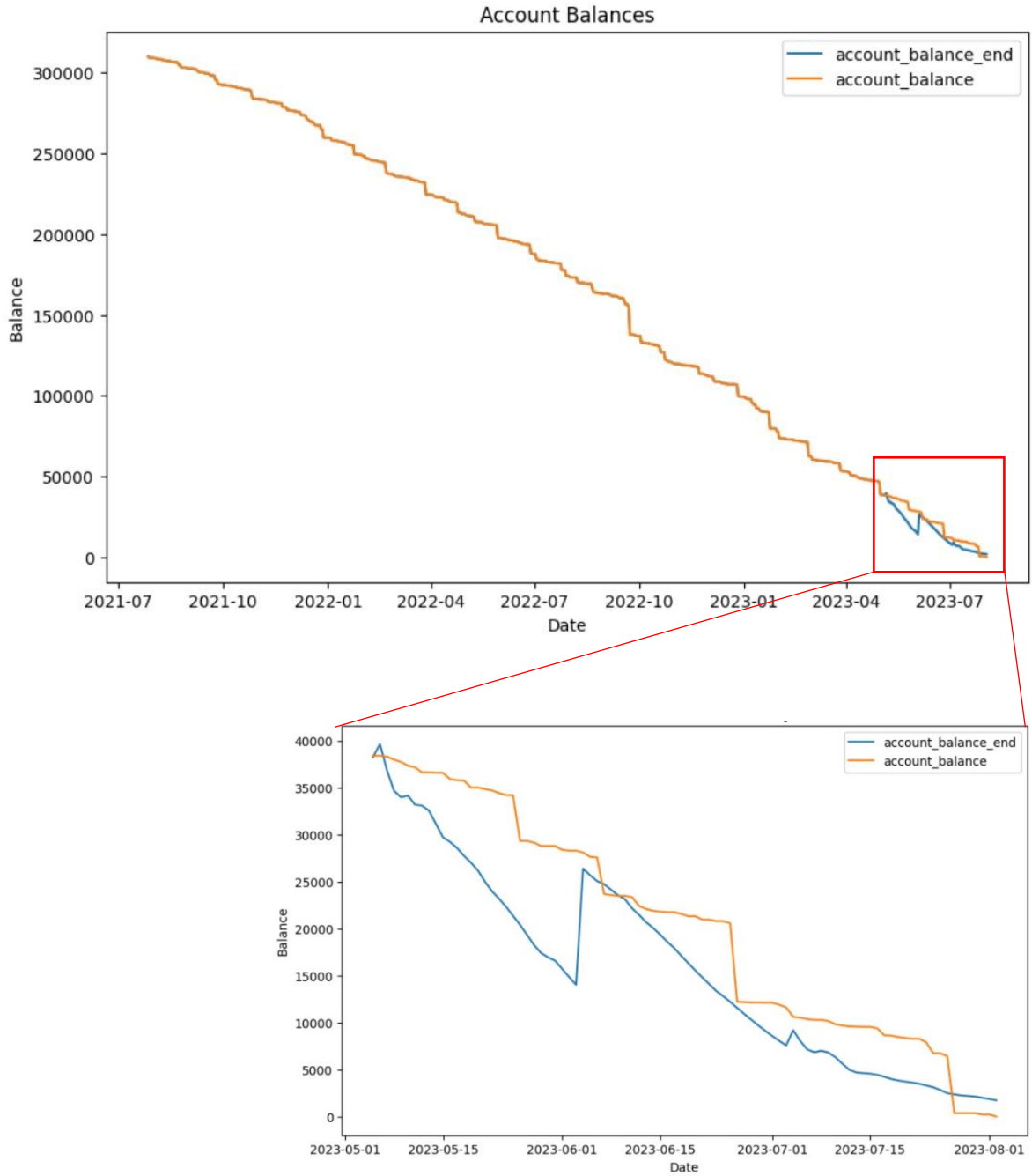
The following is a 90 day ahead balance forecast for Neural Network model:



The following is a 30 day ahead balance forecast for forecast for 30-day interval for Neural Network model:

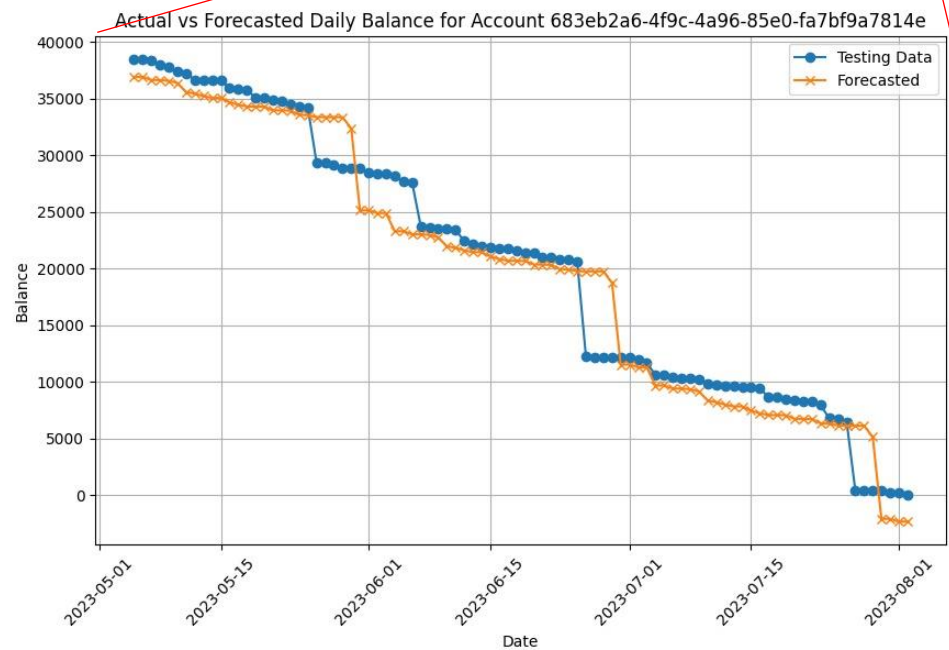
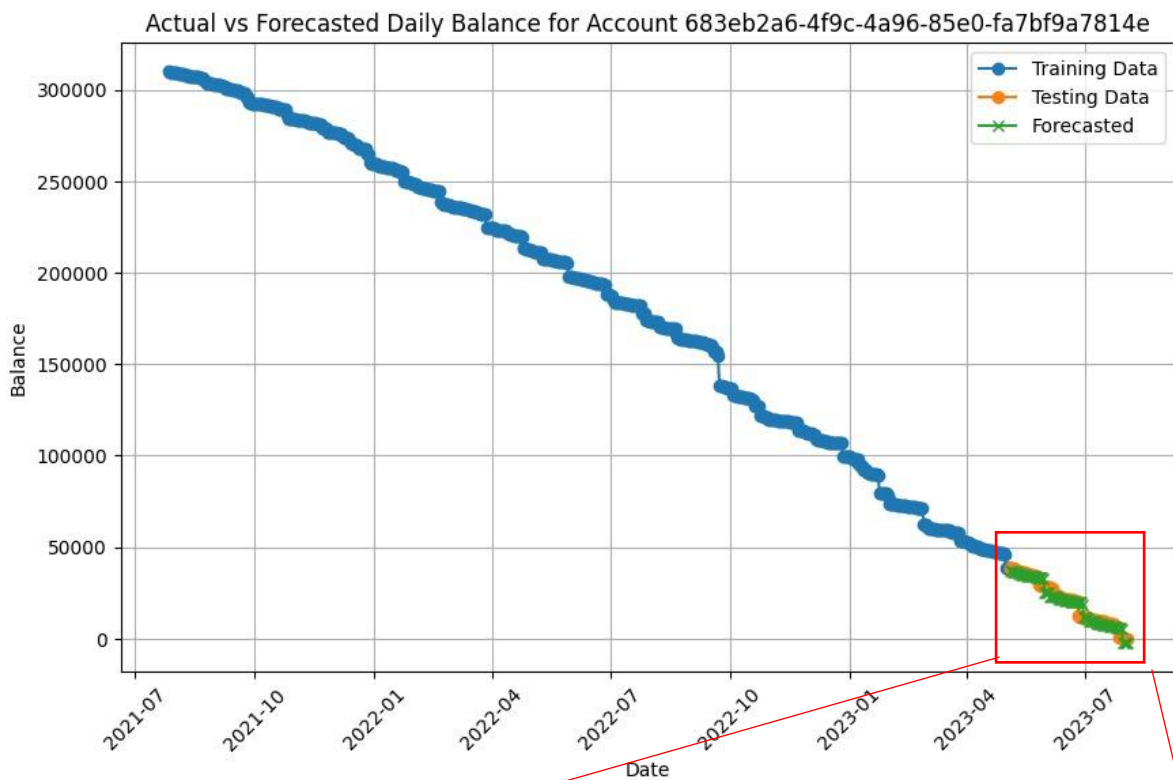


The following is a 30 day ahead balance forecast for 90-day interval for Neural Network model:

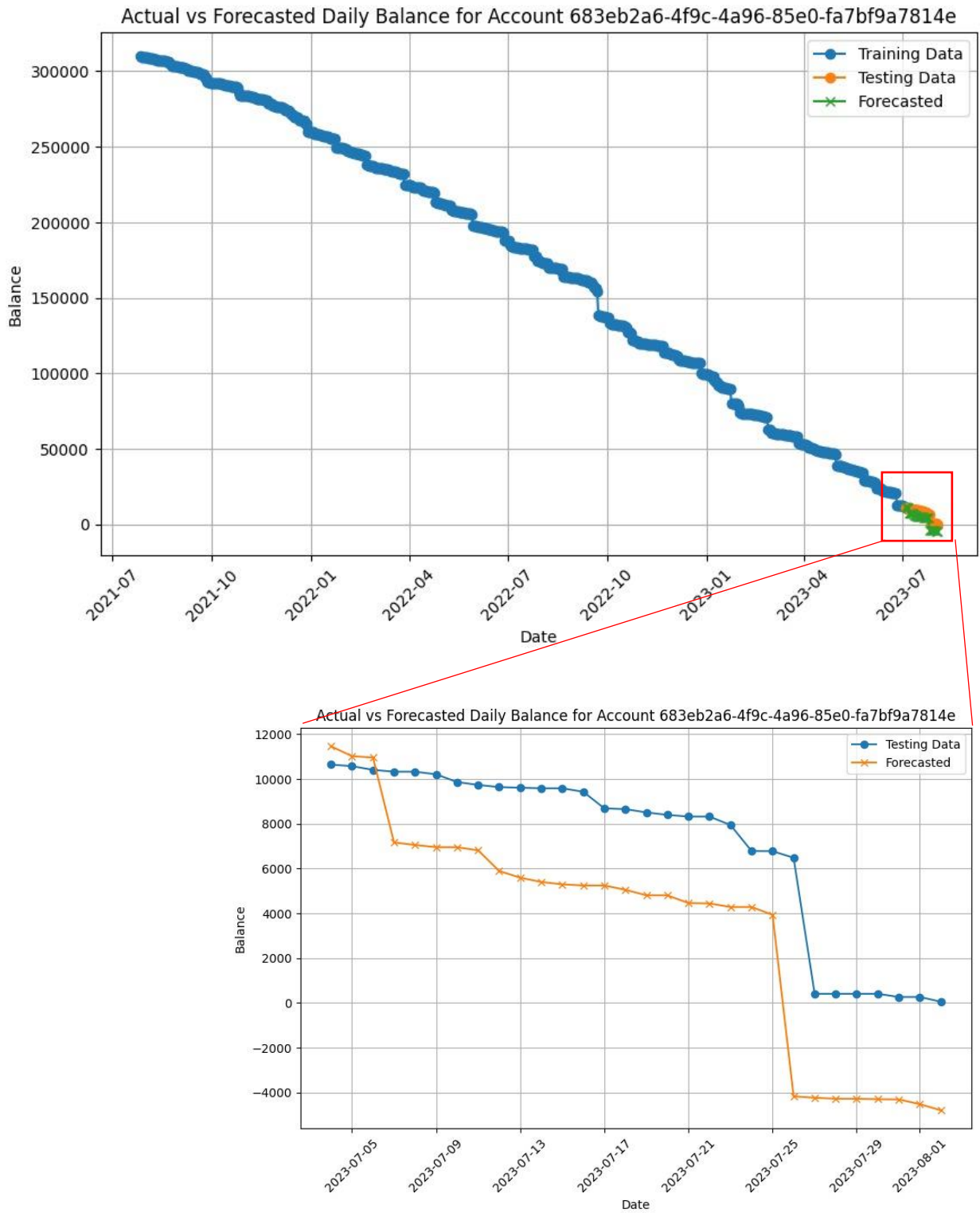


The rmse values obtained for 30 day ahead 90 day interval, 30 day ahead and 90 day ahead forecast are 9357.84, 3732.94 and 11984.26 respectively. The 90 days forecast having larger error is expected because of larger error accumulations. However, I believe the difference between two 30 days ahead forecasts is that the first 30 days of the 90 days interval being too deviated which is in my opinion natural because of the stochasticity of Neural Network algorithm.

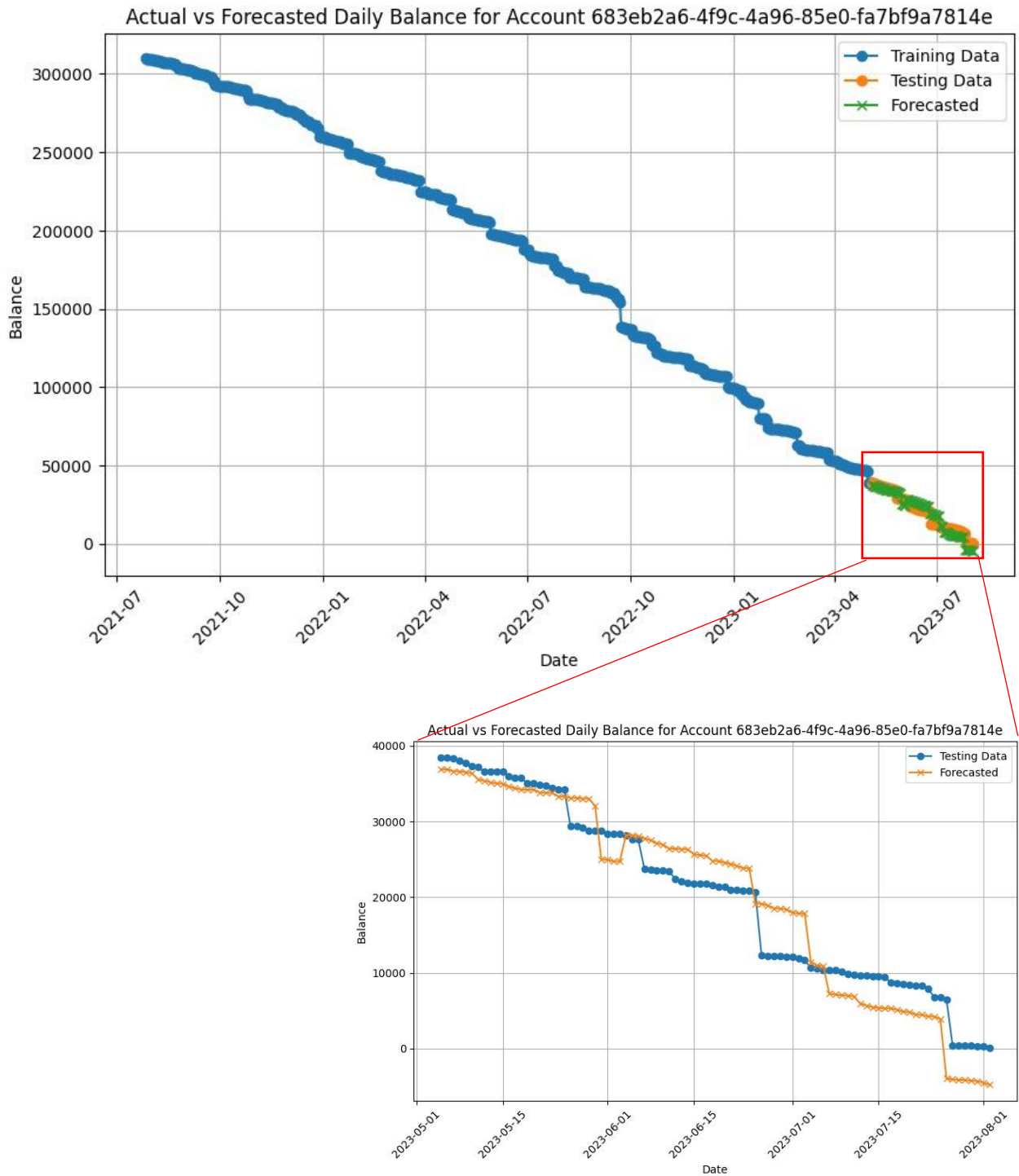
The following is a 90 day ahead balance forecast for SARIMA model:



The following is a 30 day ahead balance forecast for forecast for 30-day interval for SARIMA model:



The following is a 30 day ahead balance forecast for 90-day interval for Neural Network model:



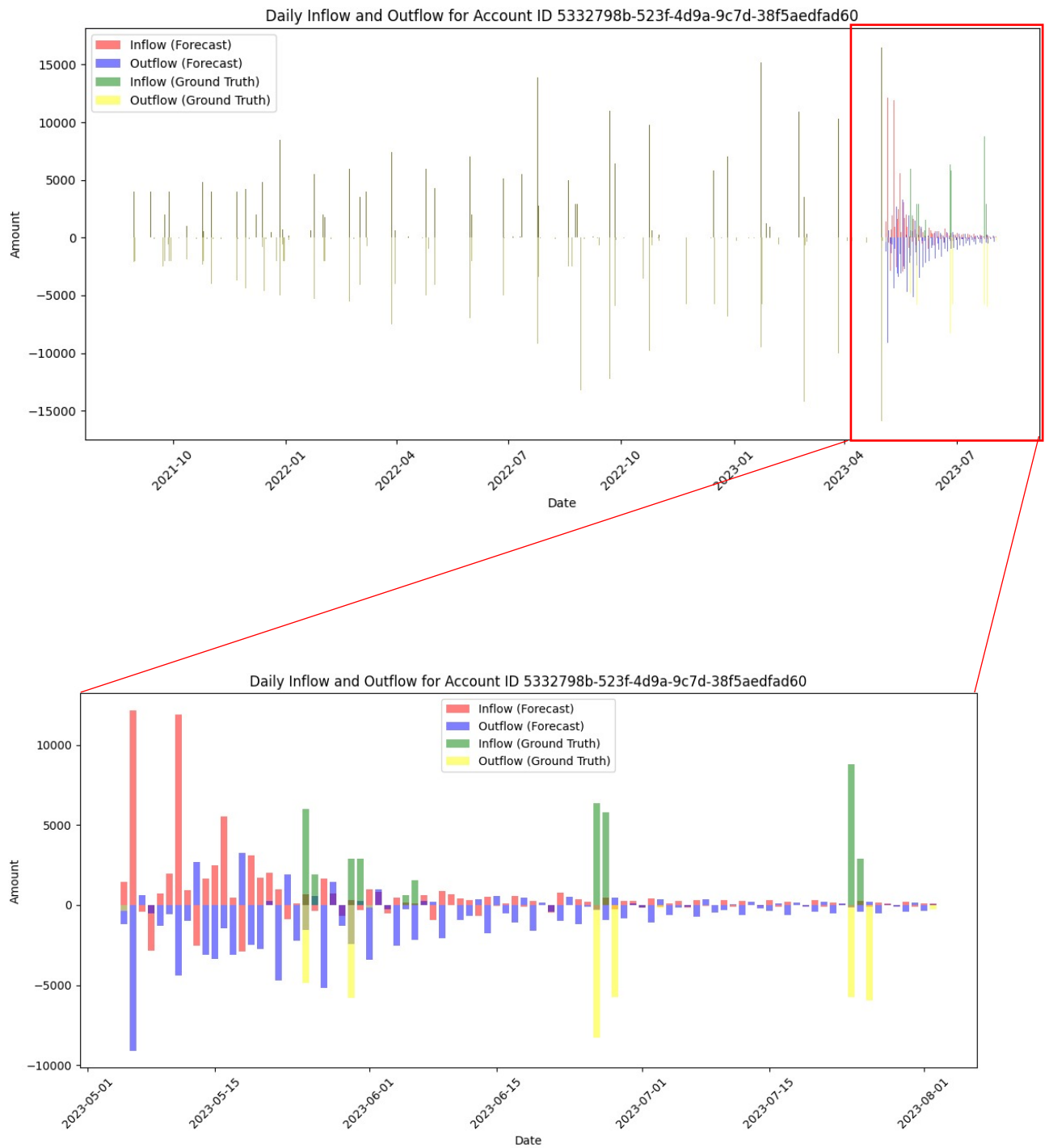
The rmse values obtained for 30 day ahead 90 day interval, 30 day ahead and 90 day ahead forecast are 4126.44, 2614.24 and 3652.79 respectively.

Even though the SARIMA algorithm performed better than the neural network, I believe that it is due to the fact that I used mostly features that are derived from the past values of the data

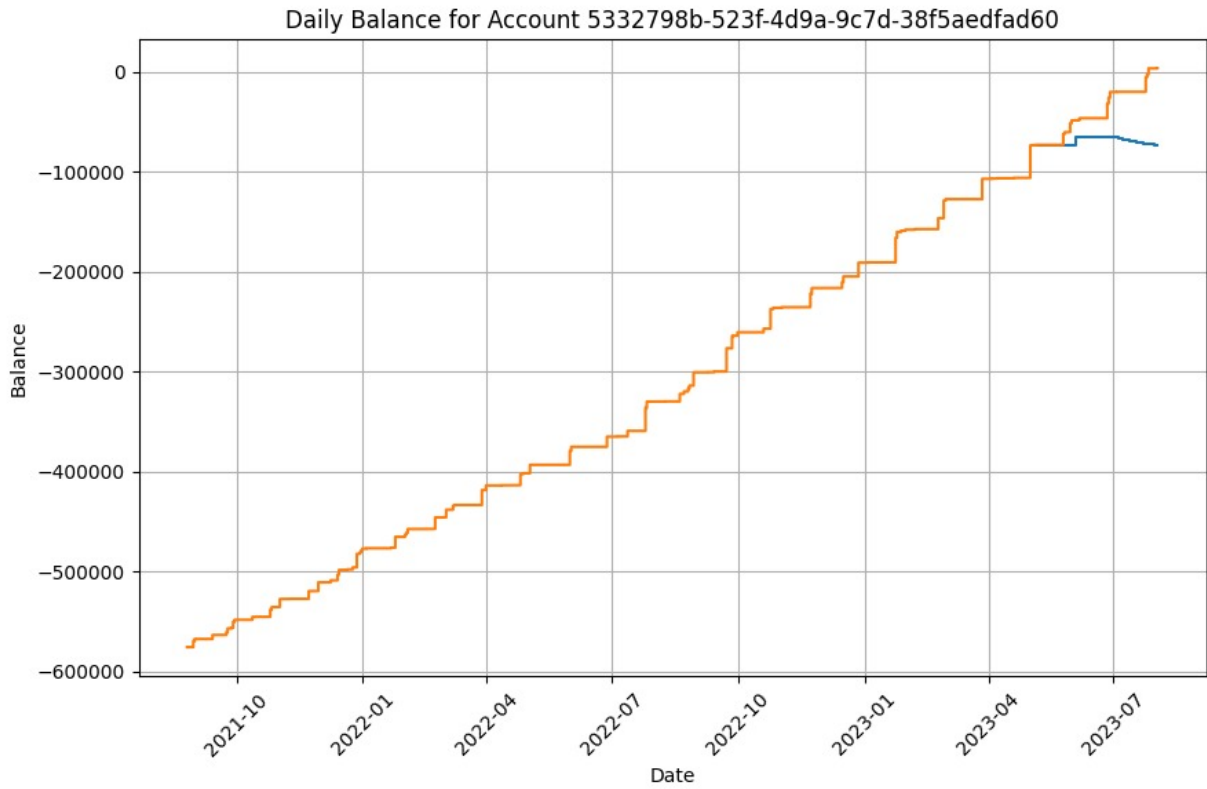
for neural network with few side features. Since SARIMA algorithm is specifically designed for autoregression it outperformed neural networks but with utilization of larger set of side features, I believe that neural networks are capable of outperforming SARIMA model in general.

Furthermore, I observed that both neural networks and SARIMA models successfully captured the trend in the data. However, SARIMA model is more successful in capturing the seasonality of the data.

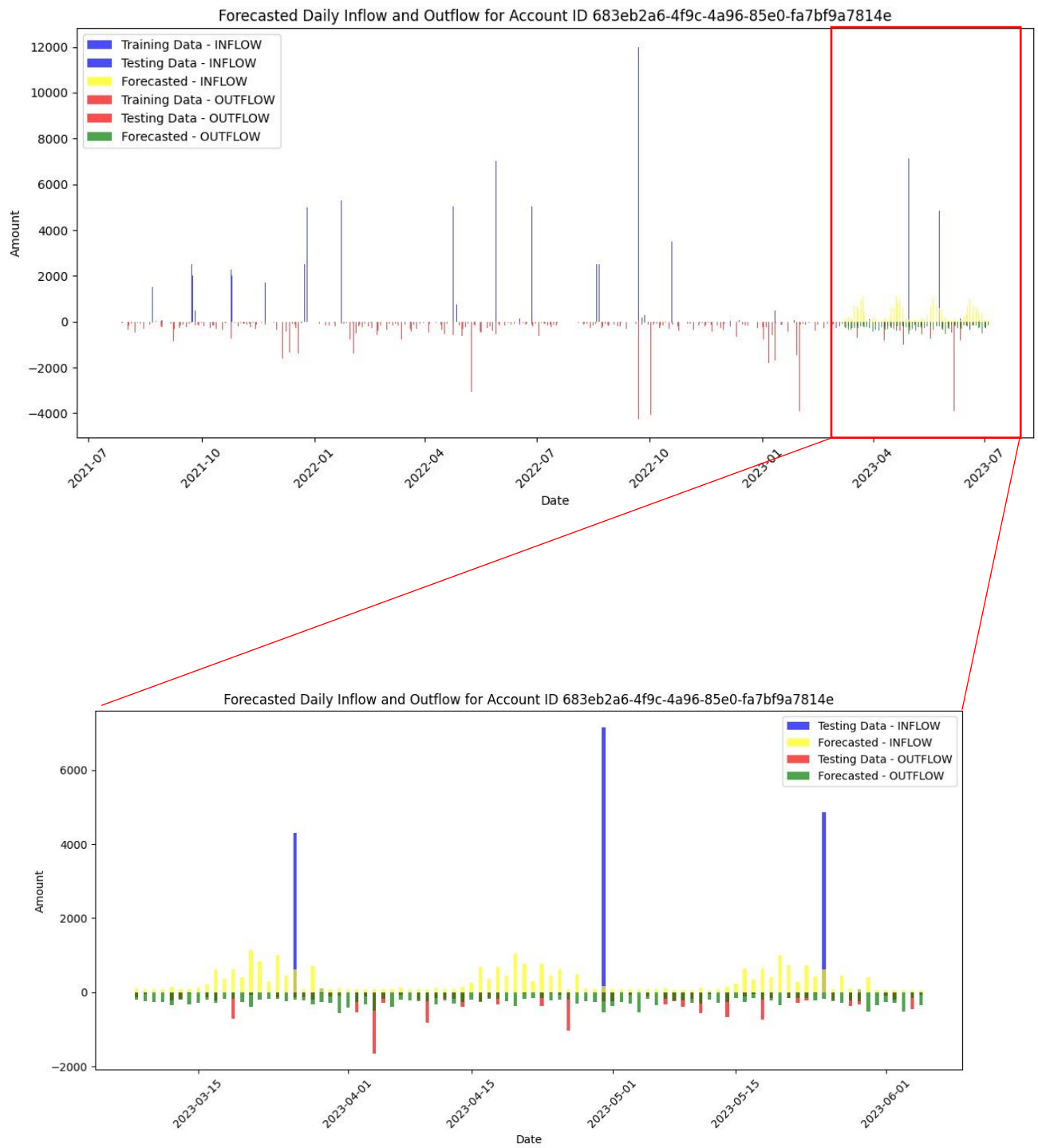
Another way to forecast the balance data is to forecast the inflow and outflow separately and then merge these forecasts. The following plot shows the inflow and outflow forecasts for 90 days together with their ground truth for Neural Network model:



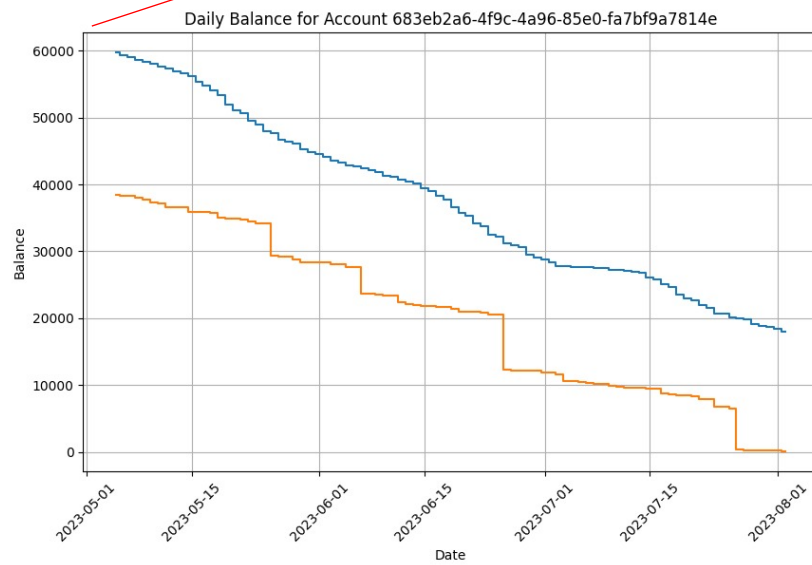
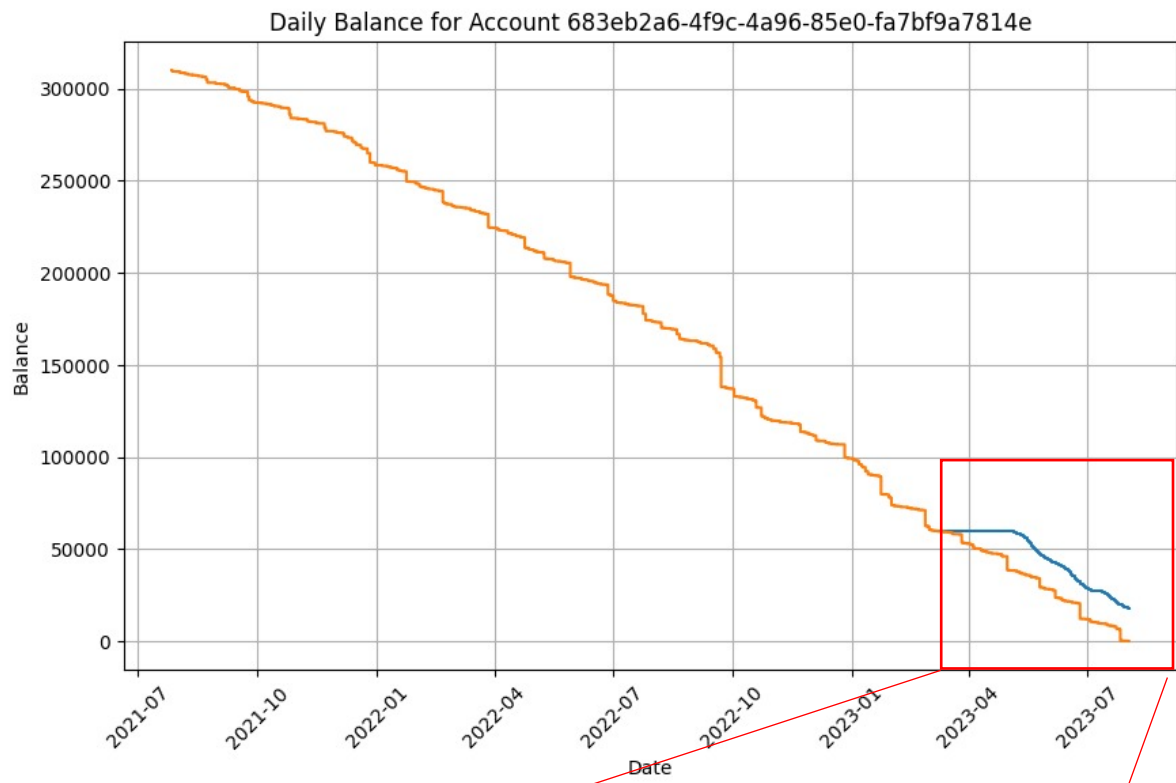
The following is 90 days ahead forecast of the balance acquired with independent forecasts of inflow and outflow with using Neural Network:



The following plot shows the inflow and outflow forecasts for 90 days together with their ground truth for SARIMA model:



The following is 90 days ahead forecast of the balance acquired with independent forecasts of inflow and outflow with using SARIMA:



It can be seen that, this type of forecast for balance yielded worse results than its direct forecast. I believe that this is due to lack of patterns in inflow and outflow graphs compared to the balance graph. Forecasting these individual values is a harder problem in terms of pattern recognition. Moreover, adding two forecasted values to make a final forecast leads to error accumulation compared to directly forecasting the balance. Hence, these results are as expected.

5.2 Future Directions:

A sensible future direction would be implementing an ensemble algorithm based on our current base algorithms and enabling online learning within the framework. The ensemble algorithm should be the final machine learning algorithm that not only determines the best performing machine learning model, but also outperforms the best performing models by using the prediction values of all the models as features. The ensemble algorithm should be based on SARIMAX and MLP. The main idea behind ensemble learning is to combine our base models to benefit their diversity and specific qualities to generate a more potent model. As an example, the SARIMAX algorithm is an algorithm that specializes in capturing linear patterns on data, while the Neural Network algorithm is used for capturing nonlinear patterns. Even though one of these models performs better on the data, both are still useful for improving the ensemble model because of the diversity they provide. We can use a state-of-the-art ensemble algorithm that utilizes meta-features and a custom loss function to yield an optimum linear combination of weights to combine each base model. After making the necessary analysis of the method, we can start implementing the model.

Since we want to compare the results of the models at the end, using an ensemble algorithm will compare results and obtain a result better than the best one concerning those results. An ensemble model is simply a machine learning algorithm fed by the outputs of several other trained models. In our ensemble implementation, we can build neural networks and LightGBM-based ensemble models separately. Each ensemble model can be fed with neural networks, and SARIMAX predictions.

The framework should also extract features from the input data, perform training with multiple models, evaluate models on the validation set, and forecast using the best performing model. The existing framework will be updated and improved accordingly for these purposes. The framework needs to be structured and work end-to-end since we want to construct an automated product with minimum initialization. The only mission of the user should be choosing a configuration with which they want to work with, and the rest of the process should be handled without the need for any intervention. I also want our framework to be generic since it should perform well with different data from different periods.

Moreover, it is also crucial that the model continuously fine-tune itself at specific periods. In other words, the model should change or expand its training dataset and perform training regularly.

6. Further Discussion:

At the beginning of the project, I got stuck reading the SQL code provided. It was functioning incorrectly even though I tried to run it in multiple database management systems. Eventually, I stopped trying to run the code on various database systems and decided to modify and debug the code. I was keep getting the following error:

```
error: exit status 3 stdout(capped at 100k bytes): DROP TYPE CREATE TYPE stderr:
NOTICE: type "public.account_accounttype_enum" does not exist, skippingERROR: function
uuid_generate_v4() does not existLINE 6: "id" uuid NOT NULL DEFAULT
uuid_generate_v4(), ^HINT: No function matches the given name and argument types. You
might need to add explicit type cast
```

In the process of refining our SQL script,I integrated two pivotal enhancements to address these specific errors, and hence, optimized the efficiency and reliability of our database operations.

Firstly, to remedy the error stating function `uuid_generate_v4()` does not exist, I incorporated the statement `CREATE EXTENSION IF NOT EXISTS "uuid-osp";`. This extension equips our PostgreSQL database with the requisite capabilities to recognize and use the `uuid_generate_v4()` function, which is instrumental in generating Universally Unique

Identifiers (UUIDs). I found out that such identifiers are essential, especially if our database schema or subsequent operations rely on UUIDs for distinguishing objects.

The second error I faced was NOTICE: type "public.account_accounttype_enum" does not exist, skipping. To tackle this, instead of making a direct attempt to drop the type which might not always exist and could potentially throw errors, I employed a more nuanced approach. I instated a conditional check to first verify the existence of the type "public.account_accounttype_enum" within the 'public' schema. Only post this verification, an attempt to drop it is made. This conditional methodology reduces the chances of disruptions by ensuring we only interact with database elements when it's completely essential. It's a strategy grounded in precision, ensuring that we're not inadvertently dropping elements, especially if they are being leveraged by other entities in the database. I have repeated this change in the code in multiple parts for example I changed the lines:

```
DROP TYPE IF EXISTS "public"."transaction_paymentchanneltype_enum";
CREATE TYPE "public"."transaction_paymentchanneltype_enum" AS ENUM ('ONLINE',
'IN_STORE', 'OTHER');
DROP TYPE IF EXISTS "public"."transaction_directiontype_enum";
CREATE TYPE "public"."transaction_directiontype_enum" AS ENUM ('INFLOW',
'OUTFLOW');
```

To:

```
DO $$
BEGIN
    IF EXISTS (SELECT 1 FROM pg_type WHERE typname =
'transaction_paymentchanneltype_enum' AND typnamespace = (SELECT oid FROM
pg_namespace WHERE nspname = 'public'))
    THEN
        DROP TYPE public.account_accounttype_enum;
    END IF;
END $$;

CREATE TYPE "public"."transaction_paymentchanneltype_enum" AS ENUM ('ONLINE',
'IN_STORE', 'OTHER');
```

After handling that error, I got the following error:

```
error: exit status 3 stdout(capped at 100k bytes): CREATE EXTENSION DO CREATE TYPE
CREATE TABLE CREATE TABLE CREATE TABLE DO CREATE TYPE DO CREATE
TYPE CREATE TABLE CREATE TABLE CREATE TABLE CREATE TABLE CREATE
TABLE CREATE TABLE INSERT 0 8 INSERT 0 5 INSERT 0 4 INSERT 0 484 INSERT 0
16 INSERT 0 484 INSERT 0 16 INSERT 0 483 INSERT 0 17 INSERT 0 484 INSERT 0 16
INSERT 0 484 INSERT 0 16 INSERT 0 484 INSERT 0 16 INSERT 0 90 INSERT 0 144
INSERT 0 500 INSERT 0 73 INSERT 0 134 INSERT 0 96 INSERT 0 28 stderr: ERROR:
relation "public.organization" does not exist
```

This error and a thorough review of our database schema and inter-table relationships highlighted several foreign key constraints referencing the "public.organization" table. However, on closer examination, it became evident that the "public.organization" table was neither present in our current script nor in the existing database structure.

To ensure that our SQL script runs without hindrance and to avoid potential inconsistencies or errors, I decided to remove all lines referencing the "public.organization" table. It's essential to understand that the presence of foreign key constraints referencing a non-existent table would lead to errors upon execution, thereby disrupting the database setup process. I have followed a similar approach on modification of : “ALTER TABLE "public"."account" ADD FOREIGN KEY ("org_id") REFERENCES "public"."organization"("id");” and similar lines.

I have spent a lot of time on this process and believe that the initially given code does not work properly. Hence, I wanted to allocate a section for explaining this process and the errors I faced during that time. I thought that instead of asking about this error handling it myself would be better in terms of showing my approach on such errors.